



**POLYTECHNIQUE
MONTREAL**

UNIVERSITÉ
D'INGÉNIERIE

INF3500 - Conception et réalisation de systèmes numériques

Laboratoire 1 Vérification de code-barres

Florian DELHON

Hiver 2026

Introduction

À la fin de ce laboratoire, vous devrez être capable de :

- Concevoir un module VHDL combinatoire, à partir de ses spécifications
- Concevoir un banc d'essai VHDL pour valider ce module combinatoire
 - Utiliser les différents types VHDL
 - Utiliser les assertions (énoncé `assert/report`)
- Effectuer la validation matérielle sur FPGA, après synthèse, implémentation et programmation

Ce laboratoire s'appuie sur et complète le matériel suivant :

1. Concepts couverts dans le **Laboratoire 0**
2. Matière du cours « **Thème 02 - Technologies de logique programmable** »
3. Matière du cours « **Thème 03 - Circuits combinatoires** »

Liste des fichiers

Fichier	Contenu
src/code_barre.vhd	Module de détection de validité de code-barres
src/code_barre_tb.vhd	Banc d'essai du module <code>code_barre</code>
src/top.vhd	<i>Top level</i> du projet
xdc/PYNQ-Z2-v1.0.xdc	Fichier de contraintes pour la PYNQ-Z2
synth-impl/create_zynq_wrapper.tcl synth-impl/zynq.tcl	Scripts <i>wrappant</i> le <i>top level</i> dans un design contenant les connexions nécessaires avec le processeur ARM
tutoriels/	Dossier contenant des Notes qui vous seront utiles dans la réalisation des laboratoires

Séance de laboratoire

Une séance de 3 heures sera dédiée à la réalisation de ce laboratoire : le **29 janvier 2026 pour le groupe 2**, et le **5 février 2026 pour le groupe 1**. Elle aura lieu à **8h30**, dans la salle **L-3714**.

Remise de code

Une remise de votre code est demandée sur votre dépôt **GitHub**. Pour ce faire, un *push* final contenant l'ensemble des fichiers nécessaires doit être fait avant le **11 février 2026 à 22h, pour les deux groupes**. Vous êtes libre de faire les *push* intermédiaires souhaités avant cette date. Il vous est demandé de **ne pas modifier les noms des fichiers/entités/architectures, ni la liste des generic/port**.

Entrevue/démonstration

Votre compréhension des travaux effectués dans ce laboratoire sera évaluée lors d'**une entrevue en présentiel**. Elle se tiendra au cours de la séance dédiée au laboratoire 2 : le **12 février 2026 pour le groupe 2**, et le **19 février 2026 pour le groupe 1**. Bien que l'entrevue se déroule en binôme, il sera demandé que **les deux élèves répondent de manière équilibrée** aux questions posées. Un élève n'ayant pas démontré une compréhension suffisante lors de l'entrevue pourra être pénalisé sur sa note individuelle.

Barème

Ce laboratoire sera noté sur 20, et comptera pour 4% de votre note finale.

Critères	Points
Remise de code	10
Partie 1 • Votre module <code>code_barre.vhd</code> fonctionnel	3
Partie 2 • Votre banc de test <code>code_barre_tb.vhd</code> complet	4
Partie 3 • Votre <i>bitstream</i> <code>top.bit</code> fonctionnel	2
Partie 4 [Défi] • Votre affichage de la position de l'erreur (<code>code_barre.vhd</code>, <code>code_barre_tb.vhd</code> et <code>top.bit</code> modifiés)	1
Entrevue/démonstration	10
Partie 1 • Votre conception de l'architecture <code>code_barre.vhd</code>	3
Partie 2 • Votre démarche de validation comportementale (logique de vérification <code>code_barre_tb.vhd</code>, résultats de simulation)	3
Partie 3 • Votre démarche de validation matérielle (<i>notebook</i> Jupyter, démonstration sur la PYNQ-Z2)	2
Partie 4 [Défi] • Votre conception/validation pour l'affichage de la position de l'erreur • Votre étude de l'utilisation des ressources (tableau, graphique, analyse)	1 1
Total	20

! Évaluation

Pour vous aider à identifier les points évalués, vous pourrez vous fier aux encadrés rouges comme celui-ci. Veillez à bien les lire afin d'obtenir la note maximale.

Partie 0 : Préparation de l'environnement de travail

Initialisation du projet Vivado

Grâce aux sources fournies dans votre dépôt de départ, **initialisez un nouveau projet Vivado**. La procédure est détaillée dans la **Note 1**.

Partie 1 : Conception du module combinatoire

Contexte

Une compagnie développe une imprimante à code-barres à haute vitesse. Elle réalise cependant que certaines impressions ont des défauts. On vous demande de concevoir un système pour inspecter les impressions. Un code-barre est constitué d'une suite de bandes blanches (0) et noires (1) en alternance. **Un code-barre valide a au moins une bande noire, et chacune d'elle doit mesurer exclusivement 1 pixel de large**. Aucune condition ne s'applique sur les bandes blanches. Voici deux exemples de code-barres valides :

0	1	0	0	0	1	0	1
---	---	---	---	---	---	---	---

0	0	0	0	1	0	0	0
---	---	---	---	---	---	---	---

Et voici un exemple de code-barre invalide :

0	1	1	0	0	1	0	1
---	---	---	---	---	---	---	---

Conception

Développez dans le fichier `code_barre.vhd` un module permettant de vérifier la validité des code-barres imprimés. L'interface du module vous est fournie (définition de l'entity au début du fichier), et contient notamment les sorties suivantes :

- `o_erreur_encre_vide` = l'encre est vide (i.e. aucune bande noire détectée),
- `o_erreur_impression` = le code-barre est invalide,
- `o_impression_valide` = le code-barre est valide.

De plus, la compagnie demande à ce qu'un témoin LED permette de détecter en un clin d'oeil l'état des code-barres. Utilisez la première LED RGB (`o_rgb(2 downto 0)`) pour afficher le code couleur suivant :

- **rouge** = le code-barre est invalide,
- **vert** = le code-barre est valide,
- **bleu** = l'encre est vide.

Concevez une architecture adaptable à n'importe quelle longueur de code-barre (**N**).

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `code_barre.vhd` fonctionnel sur votre dépôt GitHub.
- **Pour l'entrevue/démonstration**, soyez capable de présenter votre architecture finale et d'expliquer pourquoi elle fonctionne.

Partie 2 : Conception du banc d'essai

Simulation

Utilisez l'ébauche de banc d'essai fournie dans le fichier `code_barre_tb.vhd` pour stimuler votre module. De la même manière qu'au **Laboratoire 0** :

- mettez le `code_barre_tb.vhd` en *top level* dans les « Simulation sources »,
- dans le Flow Navigator, faites « Run Simulation » et observez le chronogramme.

Évaluation automatisée des réponses

Cependant, vérifier manuellement sur le chronogramme toutes les entrées est fastidieux. Comme vu en cours, on souhaite donc mettre en place une évaluation automatisée des réponses du module, générant un message d'avertissement dès que le module ne produit pas la réponse attendue. Pour ce faire, vous devez dans votre banc d'essai :

- implémenter un algorithme générant les réponses attendues et utilisant si possible une logique différente de celle du module à tester,
- comparer ces réponses attendues avec les sorties du module pour chaque valeur d'entrée, en générant un message dès qu'elles diffèrent (énoncé `assert/report`).

Complétez en conséquence le banc d'essai fourni `code_barre_tb.vhd` pour qu'il fasse la vérification du module selon les spécifications données en **Partie 1**. Vérifiez alors que votre module fonctionne et, au besoin, corrigez votre code `code_barre.vhd`.

! Évaluation

- **Pour la remise de code**, *pushez* votre fichier `code_barre_tb.vhd` complet sur votre dépôt GitHub.
- **Pour l'entrevue/démonstration**, soyez capable de décrire en quelques mots votre logique de vérification et d'expliquer en quoi elle vous permet d'affirmer que le module remplit les spécifications, avec les résultats de simulation à l'appui.

Partie 3 : Synthèse et implémentation sur la carte

Tout au long de cette partie, n'hésitez pas à vous référer à l'énoncé du **Laboratoire 0** en cas de doute sur la marche à suivre.

Synthèse/implémentation/*bitstream*

Pour la validation matérielle, **on fixe la largeur des code-barres N à 32**. Exécutez l'ensemble du flot matériel permettant la génération du *bitstream*. En bref :

- mettez le `top.vhd` en *top level* dans les « Design sources »,
- dans le Flow Navigator, faites « Generate Bitstream » (Vivado lancera automatiquement la synthèse et l'implémentation au préalable).

Programmation/validation

Avec votre PYNQ-Z2 branchée et bootée, vérifiez le fonctionnement correct de votre système sur la carte. En bref :

- programmez la carte, en cliquant sur « Program Device » dans le « Hardware Manager »,
- connectez-vous sur le serveur Jupyter et stimulez votre design via un *notebook* similaire à celui du **Laboratoire 0**.

La sortie qui vous est renvoyée dans le *notebook* est le signal `data_out` du `top.vhd`. Regardez dans ce fichier comment ce signal est connecté aux différentes sorties du module `code_barre`.

! Évaluation

- **Pour la remise de code**, *push*ez votre *bitstream* `top.bit` fonctionnel sur votre dépôt GitHub. Il est placé dans `<project_dir>/<project_name>.runs/impl_1/`.
- **Pour l'entrevue/démonstration**, soyez capable de décrire en quelques mots votre démarche de validation dans le *notebook*, avec en point d'orgue la démonstration que votre module fonctionne sur la carte !

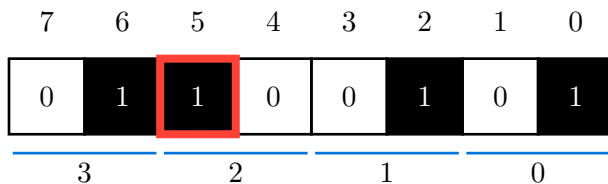
Partie 4 [Défi] : Au-delà du A

! Mise en garde

Compléter correctement les parties précédentes peut donner une note de 17/20 (85%), ce qui peut normalement être interprété comme un A. Cette dernière partie propose un défi supplémentaire pour les personnes qui souhaitent s'investir davantage dans le cours INF3500. Il n'est pas nécessaire de la compléter pour réussir le cours, ni pour obtenir une bonne note.

Afficher la position de la première erreur

La compagnie vous revient avec une spécification supplémentaire. Afin de faciliter l'inspection des code-barres défectueux, le système doit fournir la position de la première erreur dans `o_erreur_position`. **Le sens de lecture se fait du bit le moins significatif au plus significatif, donc de droite à gauche.** Par exemple, le code suivant va donner 5 :



Pour visualiser cette position en un clin d'oeil, on souhaite en plus utiliser les 4 LEDs vertes de la PYNQ-Z2 (LD0 à LD3). On divise donc la zone en 4 secteurs et indique dans la sortie `o_erreur_secteur` le secteur concerné. **Si le secteur i contient la première erreur, le i -ème bit de `o_erreur_secteur` sera à 1.** En clair, avoir `o_erreur_secteur` à 0001, 0010, 0100, ou 1000 indique que la première erreur se situe respectivement dans le secteur 0, 1, 2, ou 3. S'il n'y a aucune erreur, `o_erreur_secteur` sera à 0000.

Modifiez vos fichiers `code_barre.vhd` et `code_barre_tb.vhd` pour implémenter **et vérifier** cette nouvelle fonctionnalité. Votre architecture doit fonctionner pour n'importe quelle longueur de code-barre (N) divisible par 4.

! Évaluation

- **Pour la remise de code**, *pushez* vos fichiers `code_barre.vhd`, `code_barre_tb.vhd` et `top.bit` modifiés.
- **Pour l'entrevue/démonstration**, soyez capable de décrire très brièvement comment vous avez ajouté et validé cette fonctionnalité.

Étude de l'utilisation des ressources selon la taille N

Enfin, la compagnie vous commande une étude de la scalabilité de votre solution. Elle souhaite savoir comment l'utilisation des ressources évolue avec la taille N des code-barres à évaluer. Ici, la synthèse suffira à obtenir de bonnes estimations d'utilisation des ressources. Pour les différentes valeurs de N du Tableau 1 à la page suivante, l'objectif est donc :

- de faire la synthèse du module `code_barre` seul (et non du `top`),
- d'obtenir un rapport des ressources utilisées,
- d'extraire les métriques voulues pour remplir la ligne du Tableau 1.

On vous invite à utiliser la console TCL afin d'automatiser le processus (voir page suivante).

N	Slice LUTs	Slice Registers	F7 Muxes	F8 Muxes	Bonded IOB
8					
16					
24					
32					
48					
64					

Tableau 1. – Tableau de mesures pour votre étude de l'utilisation des ressources

i TCL dans Vivado

- `for` ([référence](#)) vous permet de créer une boucle. Par exemple, le code suivant affiche les nombres pairs inférieurs à 10 :

```
for {set i 0} {$i <= 10} {incr i 2} {
    puts "i = $i"
}
```

- `synth_design` ([référence](#)) permet de synthétiser un module. Le code suivant permet de synthétiser le module `code_barre` en attribuant `i` à `N` :

```
synth_design -top code_barre -generic N=$i -part xc7z020clg400-1 -assert
```

- `report_utilization` ([référence](#)) permet d'obtenir le rapport d'utilisation des ressources. Le code suivant permet de l'écrire dans un fichier différent pour chaque `i` (ici dans `<install-dir>/<version>/Vivado/bin/`).

```
report_utilization -file "monrapport_${i}.txt"
```

Astuce : à chaque action dans l'interface, la console TCL vous affiche les commandes exécutées. Servez-vous-en si vous souhaitez explorer le *scripting* avec TCL.

! Évaluation

- **Pour l'entrevue/démonstration**, présentez vos résultats, incluant le Tableau 1 rempli et un graphique pour l'évolution des Slice LUTs, et soyez capable de les discuter brièvement.